Structural Code Review — بيتك ومطرحك

Overview

The project follows the Flask Application Factory pattern with Blueprint separation — which is a good choice for a project of this size. Overall the structure is **correct in intent but sloppy in execution**: there are clear signs of iterative development without cleanup (leftover scripts, duplicated code, scattered imports). The review below goes file by file.

📂 Project Root — Major Clutter Problem

The root directory contains **12 leftover one-off maintenance scripts** that should never be in a deliverable project:

File	Purpose	Status
`add_views_column.py`	Manual DB migration	❌ Delete — Flask-Migrate handles this
`check_db.py`	Debug helper	❌ Delete
`clean_test_users.py`	Dev cleanup	❌ Delete
`clear_bookings.py`	Dev cleanup	❌ Delete
`delete_users.py`	Dev cleanup	❌ Delete
`fix_images.py`	Image path patch	❌ Delete
`fix_templates.py`	Template patch	❌ Delete
`init_mysql.py`	Manual DB setup	❌ Delete
`migrate_tickets.py`	Manual migration	❌ Delete
`restore_images.py`	Rollback script	❌ Delete
`seed_listings.py`	Data seeder	⚠️ Move to `app/services/`
`update_listing_images.py`	Image path patch	❌ Delete
`models_legacy.py`	Old copy of models	❌ Delete immediately

There is also a `scratch/` folder with 5 more debug/migration scripts — this entire folder should be deleted.

The root should only contain: `run.py`, `config.py`, `requirements.txt`, `.gitignore`, `README.md`

`models_legacy.py` — Dangerous Duplicate

This file is an **exact copy of** `app/models.py` with one difference: `reliability_score` defaults to `100` in the legacy version vs `0` in the active one.

Problems:

- Having two model files with different defaults is a silent inconsistency bug.
- If anyone imports from `models_legacy` by mistake, the app behaves differently.
- It's clearly an old backup left in the project.

Fix: Delete it. Use git history if you ever need to look back.

`config.py` — Good but Incomplete

```python
class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or 'dev-secret-key-beitak-w-matrak'
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or 'mysql+pymysql://...
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    UPLOAD_FOLDER = os.path.join(os.getcwd(), 'static', 'uploads')
    MAX_CONTENT_LENGTH = 16 * 1024 * 1024
```

What's good: Environment variable overrides, upload size limit.

Problems:

- No `DevelopmentConfig` / `ProductionConfig` separation — `DEBUG=True` should never be hardcoded in `run.py` for production.
- `UPLOAD_FOLDER` is defined here but **never used** in the route files. The routes hardcode `os.path.join('static', 'uploads')` themselves, ignoring this config value entirely.
- The fallback `SECRET_KEY` is a real string committed to source code. Even for dev, this is bad practice.

Fix:

```python
class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or os.urandom(32)
    UPLOAD_FOLDER = os.path.join(os.path.dirname(__file__), 'static', 'uploads')

class DevelopmentConfig(Config):
    DEBUG = True

class ProductionConfig(Config):
    DEBUG = False
```

`run.py` — Almost Empty, Has a Dead Comment

```python
if __name__ == '__main__':
    with app.app_context():
        # db.create_all()   ← dead commented code
        pass
    app.run(host='0.0.0.0', debug=True, port=8888)
```

Problems:

- `debug=True` is hardcoded — this must never reach production.
- The `with app.app_context(): pass` block does nothing. Remove it.
- `host='0.0.0.0'` exposes the server on all network interfaces — fine for dev, dangerous if left in production.

Fix:

```python
app = create_app()

if __name__ == '__main__':
    app.run(debug=os.environ.get('FLASK_DEBUG', 'false').lower() == 'true')
```

`app/__init__.py` — Good Structure, Two Issues

What's good: Clean factory pattern, blueprint registration, context processor for unread counts.

Problem 1 — Translation dictionaries belong in a constants file, not here:

```python
# These translation maps live inside the app factory
@app.template_filter('ar_city')
def ar_city(city):
    cities = {'Cairo': 'القاهرة', 'Giza': 'الجيزة', ...}
```

Every time someone adds a new city or gender option, they have to edit `__init__.py`. These should be in a `constants.py` or `translations.py` file.

Problem 2 — Bare `except:` in context processor:

```python
try:
    unread_count = Message.query...
    unread_notifications = Notification.query...
except:   # ← catches EVERYTHING silently
    pass
```

This swallows database errors, import errors, and any other exception silently on every page load. Use `except Exception as e` at minimum and log the error.

`app/models.py` — Clean but Missing Constraints

What's good: Clear model definitions, good use of relationships and backref names, separation of concerns between models.

Problems:

1. Amenities stored as a comma-separated string — should be a separate table:

```python
amenities = db.Column(db.String(500))  # "wifi,ac,kitchen..."
```

This makes it impossible to query "all listings with wifi" efficiently. It requires `LIKE '%wifi%'` which is slow and fragile. A `ListingAmenity` junction table would be correct.

2. No DB-level unique constraint on Favorite:

```python
class Favorite(db.Model):
    user_id = ...
    listing_id = ...
    # No unique_together!
```

A user can favorite the same listing multiple times. The model should have:

```python
__table_args__ = (db.UniqueConstraint('user_id', 'listing_id'),)
```

3. No DB-level unique constraint on Review: Same issue — a user can submit multiple reviews for the same listing.

```python
__table_args__ = (db.UniqueConstraint('listing_id', 'author_id'),)
```

4. `Transaction.type` uses the built-in name `type`:

```python
type = db.Column(db.String(20), nullable=False)
```

Shadowing Python's built-in `type` is a bad practice. Rename to `transaction_type` or `kind`.

5. `reliability_score` starts at `0` for new users: The legacy model had it at `100`. New users who haven't done anything have a 0 score displayed to everyone, which is confusing and unfair. It should default to a neutral value (e.g. 50 or 70).

📄 `app/routes/auth.py` — Structural Issues

What's good: Clean login/logout, proper role guard on `choose_role`.

Problems:

1. File upload code is copy-pasted twice inside the same file:

`register()` and `reupload_id()` both contain:

```python
```

```
from werkzeug.utils import secure_filename
import uuid, os
filename = secure_filename(file.filename)
unique_filename = str(uuid.uuid4()) + "_" + filename
static_path = os.path.join('static', 'uploads')
if not os.path.exists(static_path):
    os.makedirs(static_path)
file.save(os.path.join(static_path, unique_filename))
```

This code appears again in `listings.py`. That's **3 copies of the same upload logic**. It should be one helper function in `app/services/`:

python

```
# app/services/upload.py
def save_upload(file, subfolder=''):
    ...
    return saved_path
```

2. Lazy imports scattered everywhere:

python

```
def view_profile(user_id):
    from app.models import Booking, Listing  # ← inside function!
```

All models are already in `app.models`. Import them at the top of the file, not inside every function. There are 6 in-function imports in `auth.py` alone.

3. `choose_role` maps 'tenant' → 'student' hardcoded — `employee` is unreachable:

python

```
current_user.role = 'student' if role_choice == 'tenant' else 'homeowner'
```

The `employee` role exists in the model and compatibility logic but can never be assigned during normal registration.

4. No input validation on `register()`: `username`, `email`, `password`, and `national_id_number` are taken from the form and inserted directly into the database with no length check, format check, or empty-field validation beyond checking for the ID card file.

`app/routes/listings.py` — Longest File, Most Mixed Concerns

What's good: Good use of SQLAlchemy filtering and pagination.

Problems:

1. Compatibility score is calculated inside the route — should be a model method:

```python
score = 50
if current_user.smoker == listing.owner.smoker: score += 10
if current_user.sleep_schedule == listing.owner.sleep_schedule: score += 10
if current_user.gender == listing.gender_req or listing.gender_req == 'any': score +
compatibility = f"{score}%"
```

This business logic lives in a route. If the scoring formula ever changes, you have to find it buried in the view layer. It should be a method on `Listing` or `User`:

```python
# On Listing model:
def compatibility_with(self, user):
    score = 50
    ...
    return score
```

2. `delete_review` belongs in an admin blueprint, not listings:

```python
@listings_bp.route('/review/delete/<int:id>', methods=['POST'])
def delete_review(id):
    if current_user.role != 'admin':  # ← admin action in listings blueprint
```

This is an admin action forced into the listings file because there was no better place. Move it to `admin.py`.

3. `delete()` performs 5 separate queries without a transaction: The listing deletion manually deletes child records with 5 separate `.delete()` calls. If any one fails mid-way, the database is left in an inconsistent state. This should use SQLAlchemy cascade deletes configured at the model level:

```python
```

```
images = db.relationship('ListingImage', backref='listing', lazy=True, cascade='all,
bookings = db.relationship('Booking', backref='listing', lazy=True, cascade='all, de
```

4. Amenities filter uses `LIKE '%value%'` — inefficient:

```python
for amen in req_amenities:
    query = query.filter(Listing.amenities.contains(amen))
```

This works but is a symptom of the bad amenities data model (see `models.py` issue above).

`app/routes/booking.py` — Logic-Heavy, Reasonable Structure

What's good: Booking lifecycle is fully covered. Cancellation penalty/refund logic is well thought out.

Problems:

1. The `approve()` and `reject()` functions each call `db.session.commit()` twice:

```python
booking.status = 'confirmed'
...
db.session.commit()   # first commit

n = Notification(...)
db.session.add(n)
db.session.commit()   # second commit — notification could fail after booking is sav
```

If the second commit fails, the booking is confirmed but the tenant is never notified. Both writes should be in a single commit wrapped in a `try/except`.

2. `cancel()` is 80 lines — too long for one function: The cancellation function handles 5 different scenarios (confirmed by tenant, confirmed by owner, pending by tenant, pending by owner, pending_payment) all in one deeply nested block. Each scenario should be a private helper:

```python
```

```
def _cancel_confirmed_by_tenant(booking): ...
def _cancel_confirmed_by_owner(booking): ...
def _cancel_pending(booking, is_tenant): ...
```

3. Commission Bug (already in previous report — repeated here for completeness): Status is set to `'confirmed'` before counting confirmed bookings, so the 4% first-timer rate is never applied.

📄 `app/routes/admin.py` — Redundancy and Inconsistency

What's good: `before_request` hook for admin guard is a good pattern.

Problems:

1. `before_request` guard already runs, but 4 routes repeat the same check manually:

python

```
@admin_bp.before_request
@login_required
def is_admin():
    if current_user.role != 'admin': ...  # ← global guard

# Then later:
@admin_bp.route('/close_ticket/<int:id>')
@login_required          # ← redundant
def close_ticket(id):
    if current_user.role != 'admin':  # ← also redundant
```

Because `before_request` already handles this for all routes in the blueprint, the individual `@login_required` decorators and role checks on `close_ticket`, `reply_ticket`, `reject_user`, and `export_report` are dead code. Remove them.

2. Stats are queried twice in the same file: `dashboard()` queries `User.query.count()`, `Listing.query.count()`, `Booking.query.count()`, and `total_revenue` — and then `export_report()` runs the **exact same 4 queries** again. Extract them into a shared helper:

python

```
def get_platform_stats():
    return {
        'total_users': User.query.count(),
        'total_listings': Listing.query.count(),
        'total_bookings': Booking.query.count(),
        'total_revenue': db.session.query(...).scalar() or 0
    }
```

3. Chart data is hardcoded dummy values:

```
python

chart_data={
    'months': ['فبراير' ,'يناير', ...],
    'bookings': [12, 19, 3, 5, 2, 3],  # ← fake numbers
    'users': [5, 12, 11, 8, 15, 20]    # ← fake numbers
}
```

The admin dashboard shows charts with fake data. This is not just a cosmetic issue — if graders or users look at the admin panel charts, they're seeing fiction.

4. `approve_listing` missing `@login_required`:

```
python

@admin_bp.route('/approve_listing/<int:id>', methods=['POST'])
def approve_listing(id):  # ← no @login_required decorator!
```

The `before_request` guard covers this, but it's inconsistent with every other route in the file and confusing to read.

📄 `app/routes/main.py` — Mixed Responsibilities

What's good: Home, about, terms, privacy pages are clean.

Problems:

1. The `dashboard` route belongs in its own `dashboard.py` blueprint: The dashboard function routes to three completely different templates based on role. This is a routing concern that doesn't belong in `main.py`.

2. Notification routes belong in their own file or with the dashboard: `mark_notifications_read`, `delete_notification`, and `clear_all_notifications` are three

notification management routes crammed into `main.py`. They should be in a `notifications.py` blueprint or alongside the dashboard logic.

3. `favorites_page` belongs in `listings.py` or a `favorites.py` file: It's a listings-related feature sitting in the catch-all main file.

4. `resolve_map_link` is an API utility endpoint that should be in its own `api.py`: This is a server-side URL resolver for Google Maps links — it has nothing to do with "main" application pages.

5. `/reset_data` is a catastrophic route left in main.py (critical bug from previous report): It drops and recreates the entire database with no authentication. Should not exist in this file — or at all.

6. Lazy imports on every call:

```python
def dashboard():
    from app.models import Favorite, SupportTicket  # inside function
```

There are **8 in-function imports** across `main.py`. Move all model imports to the top of the file.

`app/routes/chat.py` — Reasonable but Has One Big Problem

What's good: Clean message read/unread tracking, good use of SQLAlchemy `or_`/`and_` for conversation queries.

Problems:

1. The entire profanity word list lives inside the route handler:

```python
def talk(user_id):
    ...
    bad_words = ['45... ,'كس امك' ,'كسمك more words...]
```

This is business logic (content moderation) buried in a route function. It should be a module-level constant or a service:

```python
```

```
# app/services/content_filter.py
BAD_WORDS = [...]
PHONE_PATTERN = r"..."
EMAIL_PATTERN = r"..."

def filter_message(content):
    ...
    return filtered_content, was_modified
```

2. `import re` is done inside the POST block — it's a standard library module:

```python
if request.method == 'POST':
    import re  # ← moved into a conditional
```

Standard library imports always go at the top of the file.

3. `list()` shadows Python's built-in:

```python
def list():  # ← shadows Python's built-in list
```

Rename to `conversations()` or `inbox()`.

4. No message length limit: A user can send a message of unlimited length. No server-side `len(content) > MAX` guard.

📄 `app/routes/support.py` — Too Thin

At 18 lines, this file only has one route. That's fine for a small project, but it means **admin reply and close actions** for support tickets are in `admin.py`, not here — which splits the ticket feature across two files. Move `close_ticket` and `reply_ticket` from `admin.py` into `support.py`, and use `@admin_required` decorator.

📄 Templates — Duplicated Profile File

There are **two profile templates**:

- `templates/auth/profile.html`

- `templates/profile.html`

They are almost identical (differ by ~10 lines of layout). The `profile.html` in the root templates folder is never actually used — the route renders `auth/profile.html`. The root copy is a leftover. Delete `templates/profile.html`.

Summary of All Structural Issues

🗑️ Files to Delete (10)

`models_legacy.py`, `templates/profile.html`, all 12 root maintenance scripts, the entire `scratch/` folder.

🔀 Code to Extract into Shared Helpers (3 patterns)

Repeated Code	Where	Should Become
File upload logic (uuid, secure_filename, makedirs, save)	`auth.py` ×2, `listings.py` ×1	`app/services/upload.py → save_file()`
Notification creation	`booking.py` ×4, `admin.py` ×2, `auth.py` ×1	`app/services/notifications.py → send_notification()`
Platform stats queries	`admin.py` dashboard + export_report	`app/services/stats.py → get_platform_stats()`

🔀 Code to Move to the Right File (4 items)

Code	Currently In	Should Be In
Compatibility score calculation	`listings.py` route	`Listing.compatibility_with(user)` method
Profanity word list + filter logic	`chat.py` route	`app/services/content_filter.py`
`delete_review` route	`listings.py`	`admin.py`
Notification + favorites + map resolver routes	`main.py`	Their own blueprints or correct existing ones

⚠️ In-Function Imports to Move to Top of File (20+)

Every `from app.models import X` inside a function body should be at the top of its file.

🏗️ Model-Level Fixes Needed (3)

Problem	Fix
No unique constraint on `Favorite(user_id, listing_id)`	Add `UniqueConstraint`
No unique constraint on `Review(listing_id, author_id)`	Add `UniqueConstraint`
No cascade deletes on Listing relationships	Add `cascade='all, delete-orphan'`

✅ What Is Structured Well

- Blueprint separation by feature is correct and clean
- App factory pattern is properly implemented
- `before_request` admin guard in `admin.py` is the right approach
- Models are cleanly separated from routes
- `services/seeder.py` shows awareness of the service layer pattern
- Template folder is organized by feature (`auth/`, `chat/`, `dashboard/`, etc.)